

GhostFaces: An Approach to Facial Recognition via Eigenfaces and Support Vector Machines

A Practical Study on Eigenface Decomposition, Grayscale Standardization, and Multi-Class Subspace Classification

Neerav Bhuyan Saptarshi Datta
Indian Statistical Institute

Project Repository: <https://github.com/neeravbhuyan/ghostfaces>

October 2026

Abstract

This report details the implementation and analysis of “GhostFaces,” a facial recognition pipeline designed to handle large-scale, unconstrained face datasets. Facial recognition from raw images suffers from the “curse of dimensionality,” making processing slow and prone to errors. We solve this by using Principal Component Analysis (PCA) via Thin Singular Value Decomposition (SVD) to extract “Eigenfaces”—the primary directions of facial variation. By projecting high-dimensional images onto these Eigenfaces, we compress faces into small, manageable feature vectors.

During the project, we diagnosed and fixed issues regarding artificial infrared coloring and sparse class representation. Our final model uses standard grayscale images, filters the dataset to identities with at least 5 images, retains 93 components (capturing 90% of the variance), and uses a Polynomial Support Vector Machine (SVM) to classify identities. We achieved a final accuracy of 28.04% across 355 unique identities. This report explains the steps taken, the mathematical foundation, the final results, and the known drawbacks of the Eigenface method.

Keywords: Facial Recognition, Eigenfaces, Thin SVD, Support Vector Machines, Dimensionality Reduction, Python.

Contents

1	Introduction	3
2	Data Pipeline and Preprocessing	3
2.1	Dataset Overview and Streaming	3
2.2	Grayscale Standardization	3
2.3	Threshold Filtering and Train-Test Split	3
3	Mathematical Implementation	4
3.1	Mean Centering	4
3.2	Thin SVD vs. Covariance Matrices	4
3.3	Dimensionality Reduction	4
4	Python Code Implementation	4
5	Final Results and Evaluation	5
5.1	Model Accuracy	5
6	Drawbacks and Limitations of Eigenfaces	5
7	Conclusion	6

1 Introduction

Recognizing faces from raw pixel data is mathematically difficult because of the sheer size of image data. If a standard face image is 112×92 pixels, its flattened array contains 10,304 values. If we treat each image as a point in a 10,304-dimensional space, the data becomes too massive to compute efficiently, a problem known as the **curse of dimensionality**. Furthermore, raw pixels are highly sensitive to small changes in lighting or facial expression.

To solve this, we use the **Eigenface method**. Instead of looking at individual pixels, this method uses linear algebra to find the overarching patterns—like lighting gradients, face width, and distinct features—that vary the most across the entire dataset. These patterns are called **Eigenfaces**. By measuring how much of each Eigenface is present in a new photo, we can represent a complex face with just a few dozen numbers (weights) rather than thousands of pixels. We then use a classification algorithm, such as a Support Vector Machine (SVM), to learn which weights belong to which person.

2 Data Pipeline and Preprocessing

2.1 Dataset Overview and Streaming

The system uses the **faceset** dataset stored on GitHub. To make the code highly reproducible and avoid large local downloads, the images are streamed directly into the computer’s RAM using Python’s **requests** and **zipfile** libraries. The dataset contains unconstrained images, meaning the faces have different lighting, slight rotations, and varying expressions.

2.2 Grayscale Standardization

In earlier stages of the project, we experimented with converting images into a false-color pseudo-infrared map (**COLORMAP_INFERNO**) to highlight features. However, we found that this process was destructive.

System Diagnostic

The Luminance Re-Quantization Issue:

Converting grayscale images into an infrared heatmap and then collapsing them back to grayscale flattened out important shadows and highlights around the nose and eyes. To fix this, our final pipeline bypasses the pseudo-infrared map entirely. We decode the images directly into single-channel grayscale arrays using OpenCV (**cv2.COLOR_BGR2GRAY**), which preserves natural facial contours for the PCA algorithm.

2.3 Threshold Filtering and Train-Test Split

A major issue with large, unconstrained face datasets is that many people only have 1 or 2 images. If a person has only one image, and that image gets sorted into the testing set, the model will be forced to guess an identity it has never seen before (an open-set problem).

To solve this, we filtered the dataset to only keep people who had **at least 5 images** (**MIN_SAMPLES = 5**). After filtering, the dataset was reduced to 3,332 images across 355 unique people. We then used a stratified 80/20 split so every person in the test set was guaranteed to be present in the training set.

3 Mathematical Implementation

3.1 Mean Centering

Before extracting Eigenfaces, we must find the “average face” of the dataset and subtract it from every image. If Γ_i is a flattened image vector of length $D = 10,304$, the mean face Ψ is:

$$\Psi = \frac{1}{M} \sum_{i=1}^M \Gamma_i \quad (1)$$

Subtracting Ψ gives us a centered face Φ_i . This ensures the algorithm focuses on what makes each face *different*, rather than the background lighting they all share.

3.2 Thin SVD vs. Covariance Matrices

Normally, PCA requires building a massive covariance matrix C to find eigenvectors. Because $D = 10,304$, this matrix would have over 100 million entries, making it too slow to compute.

Instead, we use **Thin Singular Value Decomposition (SVD)** directly on the centered data matrix A .

$$A = U\Sigma V^T \quad (2)$$

The columns of U give us the exact same Eigenfaces in a fraction of the time, without running out of memory.

3.3 Dimensionality Reduction

We sort the Eigenfaces based on their singular values (Σ), which represent how much variance (information) they capture. We keep enough top Eigenfaces to retain 90% of the total dataset variance. In our project, this resulted in $M' = 93$ Eigenfaces.

Every face is then projected onto these 93 Eigenfaces to get a small feature vector of weights (Ω). We have successfully shrunk our data from 10,304 dimensions down to just 93.

4 Python Code Implementation

The entire pipeline is programmed in Python using `numpy`, `scipy`, `cv2`, and `scikit-learn`. Below is the core logic used to prepare the data, run SVD, and train the SVM model.

Listing 1: GhostFaces Pipeline: Data Filtering, SVD, and SVM Classification

```

1 # 1. Filter out subjects with fewer than 5 photos
2 MIN_SAMPLES = 5
3 unique_labels, counts = np.unique(y_raw, return_counts=True)
4 valid_labels = unique_labels[counts >= MIN_SAMPLES]
5 valid_mask = np.isin(y_raw, valid_labels)
6 X, y = X_raw[valid_mask], y_raw[valid_mask]
7
8 # 2. Stratified Train/Test Split
9 X_train, X_test, y_train, y_test = train_test_split(
10     X, y, test_size=0.20, random_state=42, stratify=y
11 )
12
13 # 3. Mean Centering
14 global_mean_face = np.mean(X_train, axis=0)
15 X_train_centered = X_train - global_mean_face
16 X_test_centered = X_test - global_mean_face
17
18 # 4. Thin SVD & Truncation (Retaining 90% Variance)
19 A = X_train_centered.T

```

```

20 U, S, Vt = svd(A, full_matrices=False)
21
22 explained_variance_ratio = (S ** 2) / np.sum(S ** 2)
23 cumulative_variance = np.cumsum(explained_variance_ratio)
24 M_prime = np.searchsorted(cumulative_variance, 0.90) + 1 # Result: 93
25 eigenfaces = U[:, :M_prime]
26
27 # 5. Subspace Projection & Standardization
28 W_train = X_train_centered @ eigenfaces
29 W_test = X_test_centered @ eigenfaces
30
31 scaler = StandardScaler()
32 omega_train = scaler.fit_transform(W_train)
33 omega_test = scaler.transform(W_test)
34
35 # 6. Support Vector Machine (Polynomial Kernel)
36 svm_classifier = SVC(kernel="poly", degree=2, coef0=1.0, random_state=42)
37 svm_classifier.fit(omega_train, y_train)
38
39 y_pred = svm_classifier.predict(omega_test)

```

5 Final Results and Evaluation

5.1 Model Accuracy

After projecting the faces onto 93 components and standardizing the weights, the Polynomial Support Vector Machine (Degree 2) achieved a final accuracy of **28.04%**.

Table 1: Classification Performance Summary ($K = 355$ Classes, $M' = 93$)

Metric Type	Precision	Recall	F1-Score
Macro Average	0.1753	0.1822	0.1687
Weighted Average	0.2382	0.2804	0.2395

Key Insight

Understanding the Accuracy Metric:

While 28% might seem low compared to modern deep learning standards, this is a highly challenging unconstrained dataset with 355 different people. Predicting by random chance would yield an accuracy of $\approx 0.28\%$. The model is predicting at roughly 100 times better than random chance using purely linear algebraic methods on only 93 features.

6 Drawbacks and Limitations of Eigenfaces

The 28.04% performance ceiling is a known limitation of traditional PCA/Eigenface methods when applied to real-world, messy datasets. The method has several fundamental flaws:

- **High Sensitivity to Alignment:** Eigenfaces assume that pixels match up perfectly. If an eye is at pixel (40, 50) in one image, it must be at (40, 50) in all images. If a person tilts their head, the PCA algorithm treats the shifted pixels as "noise" rather than the same feature in a different location.
- **Lighting vs. Identity:** PCA is designed to find the largest variance in the data. In real-world photos, the direction of the sunlight or flash often causes more variation than

the actual shape of a person’s face. As a result, the top Eigenfaces often capture lighting artifacts instead of identity features.

- **Linear vs. Non-Linear Boundaries:** As people rotate their heads, their pixel representations change in curved, non-linear ways. Linear methods like PCA and basic SVMs struggle to separate identities that curve into each other’s spaces.

7 Conclusion

The **GhostFaces** project successfully implements a mathematical pipeline to reduce the extreme dimensionality of face images down to a highly concentrated set of features. We showed that by moving away from pseudo-infrared coloring to standard grayscale, and by ensuring the dataset had at least 5 images per person to fix the open-set problem, the algorithm could reliably perform mathematical classification.

While we successfully compressed 10,304 pixels into just 93 values and classified 355 distinct identities with 28.04% accuracy via a Polynomial SVM, the project highlights the ceiling of purely linear methods. To push facial recognition into the 90%+ range on unconstrained data, future work must look toward non-linear solutions like Fisherfaces (Linear Discriminant Analysis) or Deep Convolutional Neural Networks (CNNs), which are immune to pixel alignment and lighting issues.

Project Repository: <https://github.com/neeravbhuyan/ghostfaces>